

Java: Smart Home Energy Monitor

Implement a Java program to monitor the energy consumption and real-time load capacity of smart home appliances. The program should:

- Manage different categories of appliances (Constant Load vs. Variable Load) by extending a base class
- Compute the total energy consumption (in Watt-hours) over a specified period
- Identify the appliance with the highest energy consumption
- Trigger an overload warning (using a custom exception) if the total real-time power load of all appliances exceeds the safe limit of the smart home system.

Class Description

Complete the classes `ConstantLoadDevice` and `VariableLoadDevice` which extend the abstract class `Device`. You must also implement the class `SmartHomeService`.

1. **Device (Pre-provided abstract class):**
2. `String name`: Stores the name of the appliance.
3. `double basePower`: Stores the active power load in Watts.
4. Constructor `Device(String name, double basePower)`
5. Getters for `name` and `basePower`.
6. `abstract double calculateEnergyConsumption(int hours)`: Abstract method to calculate consumption in Watt-hours.
7. **ConstantLoadDevice:**
8. Constructor `ConstantLoadDevice(String name, double basePower)`
9. Energy consumption = `basePower * hours`.
10. **VariableLoadDevice:**
11. Constructor `VariableLoadDevice(String name, double basePower, double dutyCycle)`
12. `dutyCycle`: A value between 0.0 and 1.0 indicating the active proportion of time (e.g. 0.6 means the device runs 60% of the time).
13. Energy consumption = `basePower * hours * dutyCycle`.
14. **PowerOverloadException (Pre-provided exception):**
15. Checked exception thrown if total real-time load (sum of `basePower` of all devices) exceeds the system capacity.
16. **SmartHomeService:**

```
double calculateTotalConsumption(List<Device> devices, int hours):
```

- Calculates the sum of energy consumption (in Watt-hours) for all devices over the given duration. If `devices` is null or empty, return `0.0`.

```
Device getPeakConsumingDevice(List<Device> devices, int hours):
```

- Returns the device that consumes the highest energy over the given hours.
- If `devices` is null or empty, return `null`. If there is a tie, return the first one in the list.

```
void monitorLoad(List<Device> devices, double maxLimit) throws PowerOverloadException:
```

- Sums the real-time `basePower` of all devices.
- If this sum exceeds `maxLimit`, throw `PowerOverloadException` with the message: `"Overload detected: Current load is {totalLoad}W, which exceeds limit of {maxLimit}W."`
- If the list is null, do nothing.

Returns * `double`: Total consumption in Watt-hours. * `Device`: Peak energy-consuming device. * `void`: For the load monitor method (throws exception if overloaded).

Constraints * `$0 ≤ NumberOfDevices ≤ 50`. * `$0.0 ≤ basePower ≤ 5000.0`. * `$0 ≤ hours ≤ 720`.

Input Format For Custom Testing * The first line contains an integer `$$$` representing the number of devices. * The next `$$$` lines contain device parameters: * Constant Load: `Constant {Name} {BasePower}` * Variable Load: `Variable {Name} {BasePower} {DutyCycle}` * The final line contains: `{Hours} {MaxPowerLimit}`.

Sample Case 0 Sample Input For Custom Testing

```
3
Constant Fridge 150.0
Variable AC 2000.0 0.6
Variable TV 100.0 0.3
24 2200.0
```

Sample Output

```
Total Consumption: 33120.00 Wh
Peak Device: AC
Error: Overload detected: Current load is 2250.0W, which exceeds limit of 2200.0W.
```

Explanation * Refrigerator: $150.0 \text{ W} * 24 \text{ h} = 3,600 \text{ Wh}$ * AC: $2000.0 \text{ W} * 24 \text{ h} * 0.6 = 28,800 \text{ Wh}$ * TV: $100.0 \text{ W} * 24 \text{ h} * 0.3 = 720 \text{ Wh}$ * Total Consumption = $3,600 + 28,800 + 720 = 33,120.00 \text{ Wh}$. * Peak Device: AC (28,800 Wh). * Total Power Load = $150.0 + 2000.0 + 100.0 = 2250.0 \text{ W}$. Since $2250.0 \text{ W} > 2200.0 \text{ W}$ capacity, a `PowerOverloadException` is thrown.

Boilerplate:

```
// student_workspace/Solution.java
import java.util.*;

/*
 * Implement the 'ConstantLoadDevice', 'VariableLoadDevice', and 'SmartHomeService' classes here.
 * The 'Device' and 'PowerOverloadException' classes are already defined in the Harness.
 * DO NOT use access modifiers (e.g.: 'public') in your class declarations.
 */

class ConstantLoadDevice extends Device {
    // Write your code here
}

class VariableLoadDevice extends Device {
    // Write your code here
}
```

```
class SmartHomeService {  
  // Write your code here  
}
```